Variational Autoencoder (VAE) for QR Phishing (Quishing) Detection: Project Proposal for DATA275P Project

Joe Nguyen
Adithya Rajendra
Anishwar Chakraborty
Atharva Deshpande
Swaroop Veerla

I. Background and Motivation

QR codes now link the physical and digital worlds, and attackers exploit this by embedding malicious URLs in them (quishing). The Federal Trade Commission warns that scammers use QR codes to send people to spoofed sites that harvest credentials, to install malware, and to create false urgency such as unpaid delivery fees (Puig, 2023). Palo Alto Networks' Unit 42 reports about 75,000 QR code detections per day in its offline web crawlers, with 15% of these pages leading to malicious links, or over 11,000 malicious detections per day. It also reports steady growth in QR code shortener traffic (55% from the first half of 2023 to the first half of 2024, and 44% the following year) and notes that attackers use shorteners, in-app deep links and direct app downloads to stay evasive to both victims and security controls (Marty et al., 2026).

The impact is direct financial and data loss for victims, and the risk for defenders is that a QR code hides its destination from the person scanning it and that attackers design these codes to evade security controls, so detection has to work from the code itself and the URL it encodes. This project explores whether VAE models trained only on benign QR images and on benign URL strings can flag malicious codes through reconstruction error and related scores, and how they compare to trivial-feature baselines. Our main contribution is an evaluation framework that separates what a VAE learns from what trivial features already explain, using domain-grouped splits, feature-based floors and stratified reporting.

II. Related Work

Prior work on quishing detection falls into two groups, depending on whether the QR image or the URL it encodes is analyzed. URL-based approaches decode the code and classify the string. Almousa and Alsuhibany (2026) fine-tuned a BERT-based model (DistilBERT tokenizer) on URL text and periodically retrained it on newly reported malicious URLs from URLhaus. They reported 98.9% accuracy on initial testing, and the model stayed robust across retraining cycles. Their premise is that static blacklists cannot keep up with new threats. The CIC-Trap4Phish dataset paper reports F1 scores of about 0.96 to 0.99 for transformer language models on URL text and about 0.88 for a CNN on the QR images (Nejati et al., 2026). The authors also found that benign and malicious QR images are nearly indistinguishable by structural similarity (SSIM about 0.34 to 0.35).

Image-based detection works from the QR image instead of the decoded URL. Trad and Chehab (2025) analyzed the pixel structure of QR codes without decoding the payload. They generated benign and phishing codes from URLs at a fixed QR version and trained classical classifiers on the pixels. Tree ensembles performed best, with AUC around 0.91, and only a subset of pixel positions carried the discriminative signal. The authors stress that these patterns do not mean QR codes visually encode malicious intent. Rather, the models capture statistical regularities in how different payloads are translated into QR structures by the encoding process. Every approach above is supervised and needs labeled malicious examples.

Autoencoders offer a different route. They are trained on normal (benign) data only and flag inputs with high reconstruction error, on the assumption that anomalies reconstruct worse than normal data. Bouman and Heskes (2025) challenged this assumption. For PCA and linear autoencoders, they proposed that inputs arbitrarily far from the training data can still be reconstructed with zero loss. For nonlinear networks, they showed the same failure in demonstrations on 2D toy data and on MNIST with a small convolutional autoencoder. A second failure mode is interpolation, where anomalies lying between or at the border of normal classes go undetected. They also found that latent-space distance and density scores can fail. Their results depend on the random seed and on which class is treated as normal, and in many cases the autoencoders appeared to work well. As a result, good detection performance does not guarantee the absence of blind spots.

III. Data Source

We use the QR-code portion of CIC-Trap4Phish 2025 (Canadian Institute for Cybersecurity, 2025; Nejati et al., 2026). The images are synthetic renders of benign and malicious URLs, not photographs of physical codes, so each image is a deterministic encoding of its URL. The download includes the URL CSVs alongside the QR images (Table 1), which lets us build both an image model and a URL model on the same data.

class (folder)	benign	malicious
csv rows	429976	575762
png files on disk	429976	575762
csv rows without a file	0	0
files without a csv row	0	0
duplicate file names	0	0
index unique	True	True
file number equals index	True	True
folder named inside qr_path	QR_All_benign	QR_All_Malicious

Table 1. The CSVs hold 1,005,738 URLs (429,976 benign and 575,762 malicious, so 57.2% of the data is malicious), and every URL has exactly one image in its class folder. Labels come from the folder, because the malicious images are also named benign_XXXXXX.png.

IV. Modeling Methodology

We study generative models trained on benign data only and scored on benign versus malicious test data. The VAEs never see malicious data in training, and malicious data is used only to evaluate them. Only the supervised reference models train on labeled malicious items.

A. Convolutional VAE on QR images.

An encoder maps an image x to a Gaussian posterior $q(z \mid x) = \mathcal{N}\big(\mu(x), \mathrm{diag}(\sigma(x)^2)\big)$, and a decoder gives $p(x \mid z)$ over pixels. Training maximizes the evidence lower bound (ELBO): $\mathcal{L}(x) = \mathbb{E}_{q(z|x)}\big[\log p(x \mid z)\big] - \beta\,\mathrm{KL}\big(q(z \mid x) \,\|\, p(z)\big), \qquad p(z) = \mathcal{N}(0, I)$

with the reparameterization $z = \mu(x) + \sigma(x) \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$

For black and white pixels a Bernoulli likelihood is a natural choice, and we will compare it with a Gaussian (squared error) likelihood on validation data. We implement the encoder, decoder and ELBO ourselves in PyTorch. A test image is scored by reconstruction error, and we also report the negative ELBO, the KL term, and latent-space distance. The input is a 65 x 65 binary module grid. Layer sizes and the latent dimension are tuned on validation data.

B. Character-level VAE on URL strings.

The URL is a sequence of characters $x = (c_1, \ldots, c_T)$ over a vocabulary of the 95 printable ASCII symbols plus one unknown token, truncated or padded to 256 characters (from our EDA, this cuts off only 0.04% of benign and 1.0% of malicious URLs for computational feasibility). A GRU encoder maps the sequence to a Gaussian posterior $q(z \mid x) = \mathcal{N}\big(\mu(x), \mathrm{diag}(\sigma(x)^2)\big)$. An autoregressive GRU decoder, conditioned on z and on the previous characters, models $p(x \mid z) = \prod_{t=1}^{T} \mathrm{Cat}\big(c_t \mid f(z, c_{<t})\big)$, trained with teacher forcing. Training maximizes:

$$\mathcal{L}(x) = \sum_{t=1}^{T} \mathbb{E}_{q(z|x)}\big[\log p(c_t \mid c_{<t}, z)\big] - \beta\, \mathrm{KL}\big(q(z \mid x) \,\|\, p(z)\big), \qquad p(z) = \mathcal{N}(0, I)$$

with β annealed from 0 up to its target value to limit posterior collapse. A test URL is scored by its per-character reconstruction negative log-likelihood, computed with $z = \mu(x)$, and by its per-character negative ELBO. Averaging over characters keeps URL length from entering the score through summation. The KL term is logged every epoch. This model gives a second generative view of the same items and lets us compare what each modality captures.

C. Connection to the course.

Both models are latent-variable generative models trained by amortized variational inference: we optimize the ELBO with the reparameterization trick and use the learned likelihood and posterior for anomaly scoring. The project covers likelihood design, the reconstruction versus KL trade-off, posterior collapse, and evaluation of a generative model.

V. Learning and Inference Challenges

- ELBO optimization: Balancing reconstruction and KL through beta, latent size and learning rate. We log the ELBO, reconstruction term and KL term against iterations.
- Posterior collapse: The decoder can ignore the latent code, which drives the KL term toward zero. We use KL annealing and monitor the KL term each epoch.
- Scores that may not separate anomalies: Reconstruction error can miss anomalies that the model reconstructs well (Bouman & Heskes, 2025). We compare several scores and treat detection performance as something to verify.

- Shortcut features: Simple properties such as QR size, pixel density, URL length, the URL scheme or an IP-address host could explain a score without anything being learned about maliciousness. QR size follows URL length almost exactly (Figure 1 in Section VI below), 91% of benign URLs have no scheme against 37% of malicious ones, and 26% of malicious URLs have an IP-address host against 0.006% of benign ones (Figure 2). The trivial-feature floors and the stratified reporting in Section VI.b account for this.

- Input resolution: Each QR image is drawn at 6 pixels per module. We keep one pixel per module and place the grid at the top-left of a 65 x 65 canvas so the fixed corner patterns sit in the same place in every image. The canvas holds 99.9% of benign and 99.3% of malicious codes in a random sample of 3,000 images per class in EDA. The rest are excluded and counted, because cropping would cut off part of a code. Micro QR codes stay in the model and are reported as their own stratum.

VI. Preliminary Experiments

A. Exploratory Data Analysis (EDA) Results.



Figure 1. QR size by class, over-representation, and size following URL length.

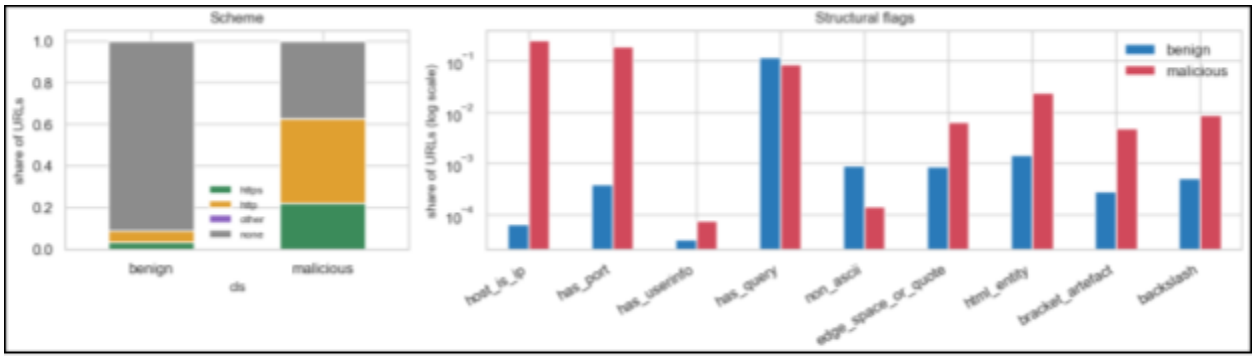


Figure 2. URL Scheme by class and the structural flags.

B. Baseline & Reference Modeling Plan.

We compare each VAE with two kinds of reference. All of them use the same splits, the same evaluation framework and the same threshold rule as the VAEs, and higher scores mean more likely malicious. A floor uses no learned image or text model: it is a Gaussian fitted to benign training items on a few easy-to-measure properties, scored by the distance from the benign mean (absolute z-score for one feature, Mahalanobis distance for several). A VAE shows

it learned something only if it beats its floor. For URLs, floor A is the headline floor with only URL length, and floor B is a stricter one with URL length, has-scheme flag, IP-host flag, number of host labels, path length, and digit ratio. For images, the floor uses QR size (modules per side) and dark-pixel fraction, and floors A and B are the same because the EDA found no further image property to add. The supervised references (an image CNN and a URL GRU classifier) reuse the matching VAE's encoder (convolutional layers & GRU) plus one linear layer for the logit and are trained from scratch with labels. They show what labels and malicious examples add, and they are context, not an upper bound.

VII. Evaluation Framework

A. Training and splits.

The VAEs and the floors are trained on benign items only. The supervised references are trained on labeled benign and malicious items. Benign and malicious rows are each split 70/15/15 into train, validation and test, grouped by the URL domain e.g., wikipedia as a dominant majority, so no domain appears in more than one split, and all models use the same splits. The largest domains fill the train first and the rest are dealt to validation and test, so validation and test measure performance on unseen sites. To answer the question “does the model catch a malicious URL it has never seen a similar one of?”, we group because, from our EDA, a random split leaves 71% of validation and test rows sharing a domain host with a training row (78% share the domain), while grouping by domain leaves 0.057%. Additionally, we remove 38 rows whose URL appears in both classes as part of data cleaning. Hyperparameters (latent size, beta, and likelihood) are tuned on validation data only. The test set is used once per model, after the configuration is fixed.

B. Scores.

For the image VAE: reconstruction error, negative ELBO, KL term and latent-space distance (Mahalanobis distance examined by Bouman & Heskes, 2025). For the URL VAE: per-character reconstruction negative log-likelihood and per-character negative ELBO, plus KL per epoch as a collapse diagnostic.

C. Metrics & Seeds.

AUROC, AUPRC, and recall at 1% and 5% false-positive rate. The threshold for each FPR is set on validation benign data only and then applied to the test set. Every split is about 57% malicious, so a random scorer has an AUPRC of about 0.57, and we report this prevalence next to AUPRC. We give bootstrap 95% confidence intervals for every metric, resampling whole domains. Latent size, beta, and likelihood (Bernoulli versus Gaussian) are varied one at a time on validation data, the chosen configuration is then run with at least 5 seeds, reporting the mean and spread.

AUROC is also reported within strata: QR version for the image VAE (Micro QR codes as their own stratum), and URL-length bin and URL scheme for the URL VAE, to test whether detection depends on image size or on how the URL is written. From our EDA, almost no

benign URLs have an IP-address host, so that group is reported as a share and no AUROC is computed for it.

D. Success criteria.

These are fixed on validation data before the test set is touched, and are stated separately for each VAE. Not meeting these criteria is a finding, not a failure. Meeting them shows a VAE uses more than its floor's features and does not by itself show what the model has learned.

- **Beats its floor:** On the test items, a paired bootstrap of the AUROC difference (VAE minus its own floor A) has a 95% CI above zero. Floor B is reported alongside it.
- **Holds within strata:** The same holds in a majority of strata that contain enough items for a stable AUROC, and every stratum is reported. For the image VAE, the strata are QR versions, with Micro QR as its own stratum. For the URL VAE, they are URL-length bins and URL schemes.
- **Seed robustness:** The lowest of the at least 5 seeds still beats the floor, since Bouman & Heskes (2025) report that results depend on the seed.
- **Gap to the supervised reference (descriptive, no threshold):** The fraction of the floor-to-reference AUROC gap the VAE closes, (VAE minus floor) / (reference minus floor).

VIII. References

Almousa, H., & Alsuhibany, S. A. (2026). QR code security: An adaptive retraining approach for dynamic URL-based threat detection. Scientific Reports, 16, Article 13143. https://doi.org/10.1038/s41598-026-43002-z

Bouman, R., & Heskes, T. (2025). Autoencoders for anomaly detection are unreliable. arXiv. https://arxiv.org/abs/2501.13864

Canadian Institute for Cybersecurity. (2025). CIC-Trap4Phish 2025 [Data set]. University of New Brunswick. https://www.unb.ca/cic/datasets/trap4phish2025.html

Marty, D.-O., Farooqi, S., & Starov, A. (2026, February 13). Phishing on the edge of the web and mobile using QR codes. Unit 42, Palo Alto Networks. https://unit42.paloaltonetworks.com/qr-codes-as-attack-vector/

Nejati, F., Rabbani, M., Eskandarian, M., Mirani, M., Piya, G., Opushnyev, I., Ghorbani, A. A., & Dadkhah, S. (2026). CIC-Trap4Phish: A unified multi-format dataset for phishing and quishing attachment detection. arXiv. https://arxiv.org/abs/2602.09015

Puig, A. (2023, December 6). Scammers hide harmful links in QR codes to steal your information. Federal Trade Commission, Consumer Advice.

https://consumer.ftc.gov/consumer-alerts/2023/12/scammers-hide-harmful-links-qr-codes-steal-your-information

Trad, F., & Chehab, A. (2025). Detecting quishing attacks with machine learning techniques through QR code analysis. arXiv. https://arxiv.org/abs/2505.03451